

Lesson 6: Exercises - Polio Case Study

Aaron A. King Edward L. Ionides Translated in pypomp by
Kunyang He

2025-12-24

Table of contents I

1 Setup

- Import Packages
- Load Data and Construct Model

2 Exercise 1: Initial Values

- Problem Statement
- Solution

3 Exercise 2: Starting Values

- Problem Statement
- Solution

4 Exercise 3: Demography

- Problem Statement

Table of contents II

- Solution

5 Exercise 4: Diagnosing Filtering Problems

- Problem Statement
- Solution

6 Exercise 5: Algorithmic Parameters

- Problem Statement
- Solution

7 Summary

- Key Insights from Exercises

8 Acknowledgments and License

9 References

This document contains worked solutions to the exercises from Lesson 6 on the polio case study, implemented using **pypomp**.

Import Packages

```
import jax
import jax.numpy as jnp
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from scipy.stats import nbinom, chi2, qmc
from copy import deepcopy

import pypomp as pp

np.random.seed(594709947)
```

Load Data and Construct Model I

```
# Load the data
data = pd.read_csv("polio_wisconsin.csv", comment='#')

# Constants
K = 6
t0 = 1932 + 4/12

# State variable names
statenames = ["SB1", "SB2", "SB3", "SB4", "SB5", "SB6", "IB", "S0", "IO"]

print(f>Data loaded: {len(data)} rows<
print(f>Time range: {data['time'].min():.2f} to {data['time'].max():.2f}<
```

Data loaded: 288 rows

Time range: 1931.08 to 1955.00

Load Data and Construct Model II

```
# Load covariates from R-generated file (covar1.csv)
# This ensures consistency with the R/pomp analysis
cov_raw = pd.read_csv("covar1.csv")

# Transpose: variable names in first column, times as column headers
first_col = cov_raw.columns[0]
covars = (cov_raw.rename(columns={first_col: "var"})
          .set_index("var")
          .T)

covars.index = covars.index.astype(float)
covars.index.name = "time"
covars = covars.sort_index()

print("Covariate columns:", ", ".join(covars.columns))
print(f"Time range: {covars.index[0]:.4f} to {covars.index[-1]:.4f}")

Covariate columns: B, xi1, xi2, xi3, xi4, xi5, xi6, P
Time range: 1931.0833 to 1955.0000
```

Load Data and Construct Model III

```
# Fixed parameters from covariate table
def get_initial_births(covars_df, t0_val):
    idx0 = np.argmin(np.abs(covars_df.index.values - t0_val))
    B_series = covars_df["B"].values
    return [float(B_series[max(0, idx0 - k)]) for k in range(6)]

SB0_list = get_initial_births(covars, t0)

fixed_params = {
    'delta': 1/60,
    'SB1_0': SB0_list[0], 'SB2_0': SB0_list[1],
    'SB3_0': SB0_list[2], 'SB4_0': SB0_list[3],
    'SB5_0': SB0_list[4], 'SB6_0': SB0_list[5],
}

# Starting parameters
params_guess = {
    'b1': 3.0, 'b2': 0.0, 'b3': 1.5, 'b4': 6.0, 'b5': 5.0, 'b6': 3.0,
    'psi': 0.002, 'rho': 0.01, 'tau': 0.001,
    'sigma_dem': 0.04, 'sigma_env': 0.5,
    'S0_0': 0.12, 'I0_0': 0.001,
    **fixed_params
}
```

Load Data and Construct Model IV

```
}
```

Load Data and Construct Model V

```
# Model functions
def rproc(X_, theta_, key, covars, t, dt):
    """Process model for polio transmission (Martinez-Bakker et al. 2015)."""
    b1, b2, b3, b4, b5, b6 = theta_['b1'], theta_['b2'], theta_['b3'], \
        theta_['b4'], theta_['b5'], theta_['b6']
    psi, sigma_dem, sigma_env, delta = theta_['psi'], theta_['sigma_dem'], \
        theta_['sigma_env'], theta_['delta']

    SB1, SB2, SB3, SB4, SB5, SB6 = X_['SB1'], X_['SB2'], X_['SB3'], \
        X_['SB4'], X_['SB5'], X_['SB6']
    IB, S0, IO = X_['IB'], X_['S0'], X_['IO']

    B, P = covars['B'], covars['P']
    xi1, xi2, xi3 = covars['xi1'], covars['xi2'], covars['xi3']
    xi4, xi5, xi6 = covars['xi4'], covars['xi5'], covars['xi6']

    beta = jnp.exp(b1*xi1 + b2*xi2 + b3*xi3 + b4*xi4 + b5*xi5 + b6*xi6)
    lambda_bar = beta * (IO + IB) / P + psi
    var_epsilon = sigma_dem**2 / jnp.maximum(lambda_bar, 1e-12) + sigma_env**2

    key, subkey = jax.random.split(key)
    epsilon = jax.lax.cond(var_epsilon < 1e-6, lambda _: 1.0,
```

Load Data and Construct Model VI

```
lambda_bar = pp.random.fast_gamma(subkey, 1.0/var_epsilon) * var_epsilon,
operand=None)

lambda_n = lambda_bar * epsilon
p = jnp.exp(-(delta + lambda_n) / 12.0)
q = (1.0 - p) * lambda_n / (delta + lambda_n)

new_SB1 = B
new_SB2 = new_SB1 * p
new_SB3 = new_SB2 * p
new_SB4 = new_SB3 * p
new_SB5 = new_SB4 * p
new_SB6 = new_SB5 * p
new_S0 = (new_SB6 + S0) * p
new_IB = (new_SB1 + new_SB2 + new_SB3 + new_SB4 + new_SB5 + new_SB6) * q
new_IO = new_S0 * q

return {
    'SB1': new_SB1, 'SB2': new_SB2, 'SB3': new_SB3,
    'SB4': new_SB4, 'SB5': new_SB5, 'SB6': new_SB6,
    'IB': new_IB, 'S0': new_S0, 'IO': new_IO
}
```

Load Data and Construct Model VII

```
def dmeas(Y_, X_, theta_, covars, t):  
    """  
    Measurement density: discretized truncated normal with numerical stability.  
    """  
    rho = theta_['rho']  
    tau = theta_['tau']  
    IO = X_['IO']  
    cases = Y_['cases']  
  
    mu = rho * IO  
    # Add base variance (1.0) to prevent distribution from being too narrow  
    sd = jnp.sqrt((tau * IO)**2 + jnp.maximum(mu, 0.0) + 1.0)  
  
    def _cdf(z):  
        return 0.5 * (1.0 + jax.scipy.special.erf(  
            (z - mu) / (jnp.sqrt(2.0) * sd)))  
  
    tol = 1e-18  
    prob = jax.lax.cond(  
        cases > 0.0,  
        lambda _: _cdf(cases + 0.5) - _cdf(cases - 0.5) + tol,
```

Load Data and Construct Model VIII

```

    lambda _: _cdf(0.5) + tol,
    operand=None
)

# Ensure probability has a lower bound to prevent log(0)
prob = jnp.maximum(prob, 1e-300)

return jnp.log(prob)

def rmeas(X_, theta_, key, covars, t):
    """Measurement simulator. Returns a dict keyed by observation name."""
    rho, tau, IO = theta_['rho'], theta_['tau'], X_['IO']
    mu = rho * IO
    sd = jnp.sqrt((tau * IO)**2 + jnp.maximum(mu, 0.0))
    z = jax.random.normal(key)
    cases_cont = mu + sd * z
    cases = jnp.where(cases_cont > 0.0, jnp.round(cases_cont), 0.0)
    return {"cases": cases}

def rinit(theta_, key, covars, t0):
    """Initial state distribution."""
    P = covars['P']

```

Load Data and Construct Model IX

```
return {  
  'SB1': theta_['SB1_0'], 'SB2': theta_['SB2_0'], 'SB3': theta_['SB3_0'],  
  'SB4': theta_['SB4_0'], 'SB5': theta_['SB5_0'], 'SB6': theta_['SB6_0'],  
  'IB': 0.0, 'IO': theta_['IO_0'] * P, 'SO': theta_['SO_0'] * P  
}
```

Load Data and Construct Model X

```
# Create Pomp object
data_filtered = data[(data['time'] > t0 + 0.01)
                     & (data['time'] < 1953 + 1/12 + 0.01)].copy()
ys = data_filtered[['cases']].copy()
ys.index = data_filtered['time']
ys = ys.loc[ys.index.intersection(covars.index)]

polio = pp.Pomp(
    ys=ys,
    theta=pp.PompParameters(params_guess),
    rinit=rinit,
    rproc=rproc,
    dmeas=dmeas,
    rmeas=rmeas,
    covars=covars,
    statenames=statenames,
    t0=t0,
    nstep=1
)

print(f"Pomp object created with {len(ys)} observations")
```

Load Data and Construct Model XI

```
Pomp object created with 249 observations
```

Load Data and Construct Model XII

```
# Set run level
run_level = 2

Np = [100, 1000, 5000][run_level - 1]
Nmif = [10, 100, 200][run_level - 1]
Nreps_eval = [2, 10, 20][run_level - 1]
Nreps_local = [10, 20, 40][run_level - 1]
Nsim = [50, 100, 500][run_level - 1]

print(f"Run level: {run_level}")
print(f"Np={Np}, Nmif={Nmif}, Nreps_eval={Nreps_eval}")
```

```
Run level: 2
Np=1000, Nmif=100, Nreps_eval=10
```

Exercise 1: Initial Values I

When carrying out parameter estimation for dynamic systems, we need to specify beginning values for both the dynamic system (in the state space) and the parameters (in the parameter space).

Tasks:

- 1 Discuss issues in specifying and inferring initial conditions, with particular reference to this polio example.
- 2 Suggest a possible improvement in the treatment of initial conditions here, code it up and make some preliminary assessment of its effectiveness.
- 3 How will you decide if it is a substantial improvement?

Discussion of Initial Conditions I

```
print("Current initial condition approach:")
print("")
print("1. SB1_0 through SB6_0: Fixed from birth data (covar1.csv)")
print("  - Assumed known from vital statistics")
print("  - Reasonable since births are well-recorded")
print("")
print("2. SO_0, IO_0: Estimated as fractions of population P")
print("  - SO_0  0.12 means 12% susceptible")
print("  - IO_0  0.001 means 0.1% infected")
print("")
print("Issues:")
print("- SO_0 and IO_0 may be poorly identified")
print("- Initial conditions far from t0 may not matter much")
print("- Correlation with other parameters (e.g., rho)")
```

Discussion of Initial Conditions II

Current initial condition approach:

1. SB1_0 through SB6_0: Fixed from birth data (covar1.csv)
 - Assumed known from vital statistics
 - Reasonable since births are well-recorded
2. S0_0, I0_0: Estimated as fractions of population P
 - S0_0 0.12 means 12% susceptible
 - I0_0 0.001 means 0.1% infected

Issues:

- S0_0 and I0_0 may be poorly identified
- Initial conditions far from t0 may not matter much
- Correlation with other parameters (e.g., ρ)

Improvement: Estimate More Initial States I

```
# Alternative: Also estimate initial baby susceptibles
# as fractions
def rinit_v2(theta_, key, covars, t0):
    """
    Modified rinit: SB values scaled by a survival fraction.
    """
    P = covars['P']
    s_factor = theta_.get('s_init', 1.0)

    return {
        'SB1': theta_['SB1_0'] * s_factor,
        'SB2': theta_['SB2_0'] * s_factor,
        'SB3': theta_['SB3_0'] * s_factor,
        'SB4': theta_['SB4_0'] * s_factor,
        'SB5': theta_['SB5_0'] * s_factor,
        'SB6': theta_['SB6_0'] * s_factor,
        'IB': 0.0,
        'IO': theta_['IO_0'] * P,
        'SO': theta_['SO_0'] * P
    }

print("Alternative: Add 's_init' parameter to scale baby susceptibles")
```

Improvement: Estimate More Initial States II

Alternative: Add 's_init' parameter to scale baby susceptibles

Comparing Initial Condition Treatments I

```
print("How to assess if modification is substantial:")
print("")
print("1. Same number of parameters → Direct likelihood comparison")
print("    - If modification gives same number of params,")
print("        compare maximized log-likelihoods directly")
print("")
print("2. Different number of parameters → Use AIC")
print("    -  $AIC = -2 * \logLik + 2 * k$ ")
print("    - Lower AIC is better")
print("    - Penalizes additional parameters")
print("")
print("3. Profile likelihood")
print("    - Check if new params are identifiable")
print("    - Examine confidence intervals")
```

Comparing Initial Condition Treatments II

How to assess if modification is substantial:

1. Same number of parameters → Direct likelihood comparison
 - If modification gives same number of params, compare maximized log-likelihoods directly
2. Different number of parameters → Use AIC
 - $AIC = -2 * \logLik + 2 * k$
 - Lower AIC is better
 - Penalizes additional parameters
3. Profile likelihood
 - Check if new params are identifiable
 - Examine confidence intervals

Exercise 2: Randomized Starting Values I

Think about possible improvements on the assignment of randomized starting values for the parameter estimation searches.

Tasks:

- 1 Propose and try out a modification of the procedure.
- 2 Does it make a difference?

Latin Hypercube Sampling I

```
def lhs_params_from_box(box, n, fixed_params):  
    """  
    Generate n parameter sets using Latin Hypercube Sampling.  
    Better space coverage than simple random sampling.  
    """  
  
    param_names = list(box.keys())  
    d = len(param_names)  
  
    sampler = qmc.LatinHypercube(d=d, seed=42)  
    sample = sampler.random(n=n)  
  
    params_list = []  
    for i in range(n):  
        params = {}  
        for j, name in enumerate(param_names):  
            lo, hi = box[name]  
            params[name] = lo + sample[i, j] * (hi - lo)  
        params.update(fixed_params)  
        params_list.append(params)  
  
    return params_list
```

Latin Hypercube Sampling II

```
box = {  
    'b1': (-2, 8), 'b2': (-2, 8), 'b3': (-2, 8),  
    'b4': (-2, 8), 'b5': (-2, 8), 'b6': (-2, 8),  
    'psi': (0, 0.1), 'rho': (0, 0.1), 'tau': (0, 0.1),  
    'sigma_dem': (0, 0.5), 'sigma_env': (0, 1),  
    'SO_0': (0, 1), 'IO_0': (0, 0.01)  
}  
  
lhs_starts = lhs_params_from_box(box, 5, fixed_params)  
print(f"Generated {len(lhs_starts)} LHS starting points")
```

Generated 5 LHS starting points

Comparing Sampling Methods I

```
def uniform_params_from_box(box, n, fixed_params):
    params_list = []
    for _ in range(n):
        params = {}
        for name, (lo, hi) in box.items():
            params[name] = np.random.uniform(lo, hi)
        params.update(fixed_params)
        params_list.append(params)
    return params_list

np.random.seed(42)
uniform_starts = uniform_params_from_box(box, 20, fixed_params)
lhs_starts = lhs_params_from_box(box, 20, fixed_params)

fig, axes = plt.subplots(1, 2, figsize=(5, 2.5))

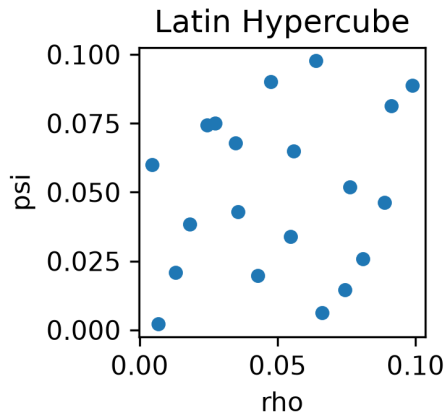
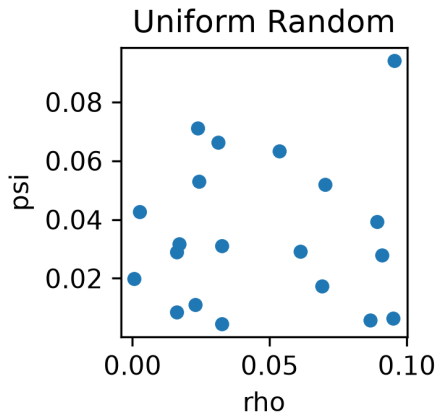
uni_rho = [p['rho'] for p in uniform_starts]
uni_psi = [p['psi'] for p in uniform_starts]
axes[0].scatter(uni_rho, uni_psi, s=20)
axes[0].set_xlabel('rho')
axes[0].set_ylabel('psi')
axes[0].set_title('Uniform Random')
```

Comparing Sampling Methods II

```
lhs_rho = [p['rho'] for p in lhs_starts]
lhs_psi = [p['psi'] for p in lhs_starts]
axes[1].scatter(lhs_rho, lhs_psi, s=20)
axes[1].set_xlabel('rho')
axes[1].set_ylabel('psi')
axes[1].set_title('Latin Hypercube')

plt.tight_layout()
plt.show()
```

Comparing Sampling Methods III



Does It Make a Difference? I

```
print("Comparison of sampling strategies:")
print("")
print("1. COVERAGE:")
print("    - LHS guarantees even coverage in each dimension")
print("    - Random sampling can have gaps and clusters")
print("")
print("2. EFFICIENCY:")
print("    - LHS often finds good regions with fewer samples")
print("    - Especially valuable in high-dimensional spaces")
print("")
print("3. REPRODUCIBILITY:")
print("    - LHS with fixed seed gives consistent coverage")
print("    - Random sampling varies between runs")
```

Does It Make a Difference? II

Comparison of sampling strategies:

1. COVERAGE:

- LHS guarantees even coverage in each dimension
- Random sampling can have gaps and clusters

2. EFFICIENCY:

- LHS often finds good regions with fewer samples
- Especially valuable in high-dimensional spaces

3. REPRODUCIBILITY:

- LHS with fixed seed gives consistent coverage
- Random sampling varies between runs

Exercise 3: Demography I

The model assumes that individuals remain in each monthly age class for one month and then move to the next.

Tasks:

- 1 In what way is this demographic model idealized?
- 2 Propose an improvement.
- 3 How would you decide if the improvement is valuable for understanding polio transmission?

Model Idealizations I

```
print("Idealizations in the demographic model:")
print("")
print("1. DISCRETE TIME:")
print("    - Assumes exact 1-month duration in each class")
print("    - Reality: continuous aging, exponential waiting times")
print("")
print("2. PERFECT SYNCHRONY:")
print("    - All births at start of month")
print("    - All transitions at month boundaries")
print("")
print("3. UNIFORM MORTALITY:")
print("    - Same death rate delta for all ages")
print("    - Reality: infant mortality higher than adults")
```

Model Idealizations II

Idealizations in the demographic model:

1. DISCRETE TIME:

- Assumes exact 1-month duration in each class
- Reality: continuous aging, exponential waiting times

2. PERFECT SYNCHRONY:

- All births at start of month
- All transitions at month boundaries

3. UNIFORM MORTALITY:

- Same death rate δ for all ages
- Reality: infant mortality higher than adults

Proposed Improvement: Gamma-distributed Duration I

```
print("Improvement: Erlang-distributed durations")
print("")
print("Instead of fixed 1-month duration, use:")
print("- Multiple sub-stages per age class")
print("- Each sub-stage has exponential waiting time")
print("- Total duration follows Erlang (Gamma) distribution")
print("")
print("Trade-off:")
print("- More realistic waiting times")
print("- But: more state variables, slower computation")
```

Improvement: Erlang-distributed durations

Instead of fixed 1-month duration, use:

- Multiple sub-stages per age class
- Each sub-stage has exponential waiting time
- Total duration follows Erlang (Gamma) distribution

Trade-off:

- More realistic waiting times
- But: more state variables, slower computation

Exercise 4: Diagnosing Filtering Problems I

Discuss the potential problems that may have arisen in the fitting of this model.

Tasks:

- 1 Check conditional log-likelihoods for outliers
- 2 Check effective sample sizes
- 3 Check parameter convergence using traces
- 4 Assess whether cooling fraction was appropriate

Setup for Diagnostics I

```
rw_sd = pp.RWSigma(
    sigmas={
        'b1': 0.02, 'b2': 0.02, 'b3': 0.02,
        'b4': 0.02, 'b5': 0.02, 'b6': 0.02,
        'psi': 0.02, 'rho': 0.02, 'tau': 0.02,
        'sigma_dem': 0.02, 'sigma_env': 0.02,
        'IO_0': 0.2, 'SO_0': 0.2,
        'delta': 0.0,
        'SB1_0': 0.0, 'SB2_0': 0.0, 'SB3_0': 0.0,
        'SB4_0': 0.0, 'SB5_0': 0.0, 'SB6_0': 0.0
    },
    init_names=['IO_0', 'SO_0']
).geometric_cooling(a=0.5)

key = jax.random.key(12345)
polio.mif(J=Np, M=Nmif, key=key, rw_sd=rw_sd)
mif_result = polio.results_history[-1]

polio.pfilter(J=Np, key=jax.random.key(99), reps=1, thresh=0, CLL=True, ESS=True)
pf_result = polio.results_history[-1]
print("Diagnostics computed")
```

Setup for Diagnostics II

Diagnostics computed

Conditional Log-Likelihoods I

```
if hasattr(pf_result, 'CLL') and not pf_result.CLL().empty:
    import plotnine as pn

    df_cll = pf_result.CLL(average=True)

    cll_mean = df_cll['CLL'].mean()
    cll_std = df_cll['CLL'].std()

    p = (
        pn.ggplot(df_cll, pn.aes(x='time', y='CLL'))
        + pn.geom_line(color='blue', size=0.5)
        + pn.geom_hline(yintercept=cll_mean, color='red', linetype='dashed')
        + pn.geom_hline(yintercept=cll_mean - 2 * cll_std,
                        color='orange', linetype='dotted')
        + pn.labs(
            x='Year',
            y='Conditional Log-Likelihood',
            title='Identifying Potential Outliers (Mean and Mean - 2*SD)'
        )
        + pn.theme_minimal()
        + pn.theme(figure_size=(6, 3))
    )
```

Conditional Log-Likelihoods II

```
print(p)

threshold = cll_mean - 3 * cll_std
outliers = df_cll.loc[df_cll['CLL'] < threshold, 'time'].values
print(f"Potential outlier times: {outliers}")
else:
    print("CLL not computed - run pfilter with CLL=True")

<ggplot: (1800 x 900)>
Potential outlier times: [1934.83333333 1946.66666667]
```

Effective Sample Size I

```
if hasattr(pf_result, 'ESS') and not pf_result.ESS().empty:
    import plotnine as pn

    df_ess = pf_result.ESS(average=True)

    p = (
        pn.ggplot(df_ess, pn.aes(x='time', y='ESS'))
        + pn.geom_line(color='green', size=0.5)
        + pn.geom_hline(yintercept=Np/2, color='red', linetype='dashed')
        + pn.labs(
            x='Year',
            y='Effective Sample Size',
            title='Particle Filter ESS'
        )
        + pn.theme_minimal()
        + pn.theme(figure_size=(6, 3))
    )
    print(p)

    low_ess = np.sum(df_ess['ESS'] < Np/10)
    print(f"Times with ESS < Np/10: {low_ess} out of {len(df_ess)}")
else:
```

Effective Sample Size II

```
print("ESS not computed - run pfilter with ESS=True")
```

```
<ggplot: (1800 x 900)>
```

```
Times with ESS < Np/10: 39 out of 249
```

Parameter Convergence I

```
if hasattr(mif_result, 'traces') and not mif_result.traces().empty:
    import plotnine as pn

    df_traces = mif_result.traces()
    params_to_plot = ['rho', 'psi', 'sigma_env', 'logLik']

    df_long = df_traces.melt(
        id_vars=['theta_idx', 'iteration'],
        value_vars=[col for col in params_to_plot if col in df_traces.columns],
        var_name='parameter',
        value_name='value'
    )

    p = (
        pn.ggplot(df_long, pn.aes(x='iteration', y='value', group='theta_idx'))
        + pn.geom_line(alpha=0.5, size=0.5)
        + pn.facet_wrap('~parameter', scales='free_y', ncol=2)
        + pn.labs(
            x='Iteration',
            y='Value',
            title='Parameter Convergence'
        )
    )
```

Parameter Convergence II

```
    + pn.theme_minimal()
    + pn.theme(figure_size=(6, 4))
  )
  print(p)
else:
  print("Traces not available")
```

```
<ggplot: (1800 x 1200)>
```

Cooling Fraction Assessment I

```
print("Cooling fraction assessment:")
print("")
print("The cooling parameter 'a' controls how quickly"
      " perturbations decrease.")
print("")
print("Signs of a being too small:")
print("  - Parameter traces flatten early (freeze)")
print("  - Log-likelihood plateaus prematurely")
print("")
print("Signs of a being too large:")
print("  - Parameters still varying at final iteration")
print("  - Need very many iterations to converge")
print("")
print(f"Current setting: a = 0.5")
```

Cooling Fraction Assessment II

Cooling fraction assessment:

The cooling parameter 'a' controls how quickly perturbations decrease.

Signs of a being too small:

- Parameter traces flatten early (freeze)
- Log-likelihood plateaus prematurely

Signs of a being too large:

- Parameters still varying at final iteration
- Need very many iterations to converge

Current setting: $a = 0.5$

Exercise 5: Choosing Algorithmic Parameters I

Suppose you have a 10-minute search with N_p , N_{mif} , and $Reps$. How would you adjust for a 2-hour search?

Scaling Strategy I

```
current = {'Np': 1000, 'Nmif': 100, 'Nstarts': 10}
time_factor = 120 / 10 # 12x

print(f"Time scaling factor: {time_factor}x")
print("")

strategies = {
    'Balanced': {
        'Np': int(current['Np'] * time_factor**(1/3)),
        'Nmif': int(current['Nmif'] * time_factor**(1/3)),
        'Nstarts': int(current['Nstarts'] * time_factor**(1/3))
    },
    'More particles': {
        'Np': int(current['Np'] * time_factor**(1/2)),
        'Nmif': current['Nmif'],
        'Nstarts': int(current['Nstarts'] * time_factor**(1/2))
    },
    'More searches': {
        'Np': current['Np'],
        'Nmif': current['Nmif'],
        'Nstarts': int(current['Nstarts'] * time_factor)
    }
}
```

Scaling Strategy II

```
}  
  
for name, params in strategies.items():  
    print(f"{name}:")  
    print(f"    Np={params['Np']}, Nmif={params['Nmif']}, "  
          f"    Nstarts={params['Nstarts']}")
```

Time scaling factor: 12.0x

Balanced:

Np=2289, Nmif=228, Nstarts=22

More particles:

Np=3464, Nmif=100, Nstarts=34

More searches:

Np=1000, Nmif=100, Nstarts=120

Two-Stage Search Implementation I

```
print("Two-stage search (recommended for 2-hour budget):")
print("")
print("Stage 1 (exploration): 1 hour")
stage1 = {'Np': 1000, 'Nmif': 50, 'Nstarts': 50}
print(f"  Np={stage1['Np']}, Nmif={stage1['Nmif']}, "
      f"Nstarts={stage1['Nstarts']}")
print(f"  Goal: Find promising regions")
print("")

print("Stage 2 (refinement): 1 hour")
stage2 = {'Np': 2000, 'Nmif': 100, 'Nstarts': 10}
print(f"  Np={stage2['Np']}, Nmif={stage2['Nmif']}, "
      f"Nstarts={stage2['Nstarts']}")
print(f"  Goal: Refine top results from Stage 1")
```

Two-Stage Search Implementation II

Two-stage search (recommended for 2-hour budget):

Stage 1 (exploration): 1 hour

Np=1000, Nmif=50, Nstarts=50

Goal: Find promising regions

Stage 2 (refinement): 1 hour

Np=2000, Nmif=100, Nstarts=10

Goal: Refine top results from Stage 1

Summary I

Exercise 1 (Initial Values):

- Initial conditions can affect likelihood but may be weakly identified
- Modifications should be compared via maximized log-likelihood
- No new parameters → direct likelihood comparison

Exercise 2 (Starting Values):

- LHS provides better coverage than uniform random
- Informed priors based on biological knowledge can help
- Multiple strategies should be compared empirically

Summary (continued) I

Exercise 3 (Demography):

- Model makes simplifying assumptions about age structure
- Continuous time with gamma-distributed durations could be more realistic
- Key: check if conclusions are robust to modifications

Exercise 4 (Diagnostics):

- Monitor ESS and conditional log-likelihoods for problems
- Check parameter traces for convergence using `traces()`
- Adjust cooling fraction if traces freeze too early/late

Exercise 5 (Algorithmic Parameters):

- More starting points generally more valuable than longer runs
- Two-stage search: explore first, refine later

Summary (continued) II

- Monitor diagnostics to guide parameter choices

Key pypomp API Points I

- 1 RWSigma requires **ALL parameters** in sigmas dict - use 0.0 for fixed params
- 2 rmeas must return a dict keyed by observation name, e.g.,
`{'cases': value}`
- 3 Use `nstep=1` for discrete-time models
- 4 Multiple starting points: pass
`theta=pp.PompParameters(list_of_dicts)` to `mif()`
- 5 Use `polio.prune(n=k)` to keep top k parameter sets
- 6 Access traces via `mif_result.traces()` (tidy DataFrame with columns `theta_idx`, `iteration`, `method`, `logLik` and `parameters`)

Acknowledgments I

- This lesson is prepared for the Simulation-based Inference for Epidemiological Dynamics module at SISMID.
- The materials build on previous versions of this course and related courses.
- Licensed under the Creative Commons Attribution-NonCommercial license (CC BY-NC 4.0).

References

